Subexponential Parameterized Approximation for Group-Fair Euclidean k-Center

Anonymous author
Anonymous affiliation

Abstract

Clustering is a basic tool for grouping data into meaningful classes, but in many applications the resulting clusters must also respect the fairness conditions among different groups. We study this direction for the group-fair Euclidean k-center problem in fixed dimension, where the input points are partitioned into disjoint color classes, and every cluster must satisfy given lower and upper bounds on the fraction of each color.

We give a separator-based recursive approximation algorithm for the continuous-center setting. The main idea is to first reduce the continuous search space to a finite candidate center set and then use the geometric sphere separators to break the instance into smaller subproblems. Our algorithm gives a bicriteria approximation: the clustering radius is within a factor of $(1+\epsilon)$ of the optimum exactly fair solution, while the fairness bounds are relaxed only by a multiplicative factor. For fixed dimension, number of colors, and approximation parameters, the running time is polynomial in input size and subexponential in k. A recent IJCAI-ECAI 2026 result [29] gives a $(1+\epsilon)$-approximation for group-fair k-center in bounded doubling metrics with running time $(k/\epsilon)^{\mathcal{O}(k)}n^{\mathcal{O}(1)}$. In the fixed-dimensional Euclidean setting considered here, we improve the dependence on k to subexponential, while allowing continuous centers. The fairness guarantees of the two results are different: the previous work allows an additive fairness violation, whereas our algorithm uses a controlled multiplicative relaxation.

2012 ACM Subject Classification Theory of computation → Facility location and clustering; Theory of computation → Parameterized complexity and exact algorithms; Theory of computation → Computational geometry; Theory of computation → Approximation algorithms analysis

Keywords and phrases Fair clustering; Group fairness; Euclidean k-center; Parameterized approximation; Bicriteria approximation; Subexponential algorithms; Geometric separators

Acknowledgements Anonymous acknowledgements

1 Introduction

Clustering is a basic tool for dividing a dataset into meaningful groups. Among the classical clustering objectives, the k-center problem aims to select at most k centers and assign every point to one of them so that the maximum assignment distance is minimized. In general metrics, the problem is NP-hard and admits a polynomial-time 2-approximation, which is optimal unless P = NP [17, 20]. Computing an optimal solution is also hard when parameterized by the number k of centers [15]. Many variants of k-center have therefore been studied, including clustering with outliers [10], anonymity constraints [2], capacities [14], lower bounds [21], and fairness constraints.

A clustering obtained only by minimizing distances may represent some demographic groups poorly. For example, a cluster may contain very few points from a minority group even when that group forms a significant part of the complete dataset. This has led to several notions of fairness in clustering, including group fairness [12, 7, 8], proportional fairness [11, 23], and individual fairness [22, 26]. In this paper, we focus on group fairness. Under group fairness, every point belongs to a demographic group, represented by a color. For every color i, lower and upper bounds α_i and β_i are given, and the proportion of color i in every nonempty cluster must lie between these bounds. Thus, fairness depends not only on the selected centers but also on the assignment of points to them. In particular, assigning every point to its nearest center may fail to produce a fair solution.

Chierichetti et al.[12] initiated the study of fair clustering for two colors and introduced the fairlet framework. A fairlet is a small group of points that already satisfies the required balance condition. Their approach first partitions the input into fairlets and then applies a standard clustering algorithm to these fairlets. The group-fairness model was later generalized to multiple colors and lower and upper proportion bounds. Bercea et al. [8] obtained a 5-approximation for exact ratio preservation and a 3-approximation under essential fairness, where the number of points of each color may differ by at most one from a fair fractional assignment. Ahmadian et al. [3] considered upper-bound representation constraints and gave a 3-approximation with a bounded additive fairness violation. Bera et al.[7] developed a more general framework that also allows protected groups to overlap. Let Δ be the maximum number of groups to which any point belongs. For fair k-center, their framework gives a 4-approximation with an additive fairness violation of at most $4\Delta + 3$. Harb and Lam [19] subsequently obtained a scalable 3-approximation while retaining the $4\Delta + 3$ violation for overlapping groups. For disjoint groups, smaller additive violations are possible. These results show that overlapping groups make the assignment problem substantially more difficult. The present paper considers disjoint color classes.

Parameterized algorithms provide another way to obtain stronger approximation guarantees. Goyal and Jaiswal [18] gave a 2-approximation for constrained k-center, including group-fair k-center, in FPT time parameterized by the number of centers and colors. They also showed that an approximation factor below 2 cannot be obtained in FPT time for general metrics under standard parameterized-complexity assumptions. Fair coresets have also been studied for the k-median and k-means objectives [5]. Moreover, even when the centers are fixed, finding a fair assignment can remain computationally difficult [16]. This explains why the assignment step cannot be treated as a simple nearest-center assignment. Several works have considered fair k-center in structured, distributed, and low-dimensional metric spaces. Ceccarello et al. [9] studied fast fair k-center algorithms in doubling metrics, while Wu et al. [28] considered the problem in a distributed setting. In fixed-dimensional Euclidean space, Moritaka et al. [25] studied fair k-center in the plane with absolute lower

and upper bounds on the number of points of each color. Their fairness requirements differ from the proportional bounds considered here.

The result closest to ours is the recent work of Wu et al. [29] on fair clustering in bounded doubling metrics. For group-fair k-center, they use the same lower- and upper-bound proportion model considered in this paper and obtain a $(1+\epsilon)$-approximation for the radius with an additive fairness violation of one. Their algorithm runs in $\left(\frac{k}{\epsilon}\right)^{\mathcal{O}(k)} n^{\mathcal{O}(1)}$ time. Their paper also studies data-summarization fairness, whereas we focus only on group fairness. Moreover, their centers are selected from the input points, while our centers may be placed anywhere in the Euclidean space.

Subexponential parameterized algorithms are already known for the ordinary Euclidean k-center problem. In particular, geometric separator methods lead to running times whose exponents involve $k^{1-1/d}$ in dimension d [1]. Closely related lower bounds are known for the discrete Euclidean problem [13]. These results motivate the question of whether geometric separators can also provide a subexponential algorithm when group-fairness constraints are present. The main difficulty is that a cluster crossing a separator may receive points of different colors from both sides, and its fairness cannot be checked independently within either side.

1.1 Our Contributions

In this paper, we study the group-fair k-center problem in fixed-dimensional Euclidean space with continuous centers. Our main contribution is a separator-based structural approach that uses the geometry of Euclidean space to reduce the search space and divide the instance into smaller subproblems while keeping only a small number of active centers.

Based on this approach, we obtain a bicriteria approximation algorithm for group-fair Euclidean k-center. Given an exactly fair solution, our algorithm returns a clustering whose radius is within a factor $(1+\epsilon)$ of the optimum exactly fair radius, while allowing a controlled multiplicative relaxation of the fairness constraints. For fixed dimension, number of colors, and approximation parameters, the algorithm runs in polynomial time in the input size with subexponential dependence on k.

▶ **Theorem 1.1.** *For any real numbers $0 < \varepsilon \leq 1$ and $\lambda > 1$, there exists a bicriteria approximation algorithm for the group-fair Euclidean k-center problem with continuous centers in fixed dimension d. The algorithm achieves a $(1+\epsilon)$-approximation for the clustering radius and a multiplicative λ-relaxation of the fairness constraints. For fixed number of colors c, dimension d, and λ, its running time is $f(k,\varepsilon)\, n^{\mathcal{O}(d)}$, where $f(k,\epsilon) = \left(\frac{k}{\epsilon}\right)^{\mathcal{O}_{c,d,\lambda}\left(k^{1-1/d}/\epsilon\right)}$.*

In particular, for fixed ϵ, the dependence on k is subexponential.

Organization of the Paper. In Section 2, we introduce the notation, preliminary concepts, and the fairness model used throughout the paper. In Section 3, we give an overview of the main technical ideas behind our separator-based approach. In Section 4, we conclude the paper and discuss possible directions for further study. Finally, in Section A, we present the algorithm and describe its main recursive steps (A.1), prove the correctness of the algorithm and establish our main bicriteria approximation result (A.2) and analyze the running time and show the subexponential dependence on k (A.3) as part of our appendix.

2 Preliminaries

For any integer $m \geq 1$, let $[m] = \{1, \ldots, m\}$. Throughout the paper, we work in the Euclidean space $\mathbb{R}^d$, where $d \geq 2$ is fixed, and let $P \subseteq \mathbb{R}^d$ denote the set of input points. For any

two points $u, v \in \mathbb{R}^d$, we write $d(u,v) = \|u - v\|_2$ for their Euclidean distance. For a point $v \in \mathbb{R}^d$ and a radius $r \geq 0$, let $B(v,r) = \{x \in \mathbb{R}^d : d(x,v) \leq r\}$ denote the Euclidean ball of radius r centered at v. For a point $v \in \mathbb{R}^d$ and a set of centers $S \subseteq \mathbb{R}^d$, we use $d(v,S) = \min_{s \in S} d(v,s)$ to denote the distance from v to its nearest center in S. For a finite set $X \subseteq \mathbb{R}^d$, let $\mathrm{diam}(X) = \max_{x,y \in X} d(x,y)$. Unlike the discrete k-center setting, the centers in our problem are continuous and may be chosen from any point of $\mathbb{R}^d$.

We use standard graph notation. For an undirected graph $G = (V, E)$, $V(G)$ and $E(G)$ denote its vertex and edge sets, respectively. A connected component is a maximal connected subgraph of G. The notation $\mathcal{O}_d(\cdot)$ means that the hidden constant may depend on d. Similarly, $\mathcal{O}_{c,d}(\cdot)$ allows dependence on c and d.

We first define the Euclidean k-center problem. Let $P \subseteq \mathbb{R}^d$ be a set of n points and let k be a positive integer. In our setting, the centers are continuous and can be chosen from anywhere in $\mathbb{R}^d$.

▶ **Definition 2.1** (Euclidean k-Center). *An instance of the Euclidean k-center problem is given by (P,k). The goal is to find a set $C \subseteq \mathbb{R}^d$, with $|C| \leq k$, such that $\max_{p \in P} d(p,C)$ is minimized, where $d(p,C) = \min_{a \in C} \|p - a\|_2$.*

We now introduce the group-fairness condition used in this paper. Suppose the input points are partitioned into c disjoint color classes $\mathcal{P} = \{P_1, \ldots, P_c\}$, such that $P_1 \cup P_2 \cup \cdots \cup P_c = P$ and $P_i \cap P_j = \emptyset$ for any $i \neq j$. Every point in P_i is said to have color i. For every color $i \in [c]$, we are given two values α_i and β_i, where $0 \leq \alpha_i \leq \beta_i \leq 1$. These values specify the lower and upper proportion of color i allowed in every cluster.

▶ **Definition 2.2** (Group-Fair Euclidean k-Center). *An instance of the group-fair Euclidean k-center problem, denoted by GF-k-CEN, is given by $(P, \mathcal{P}, \alpha, \beta, k)$, where $\alpha = (\alpha_1, \ldots, \alpha_c), \beta = (\beta_1, \ldots, \beta_c)$. The goal is to find a set $C \subseteq \mathbb{R}^d$ of at most k centers and an assignment $\phi : P \to C$ that minimizes $\max_{p \in P} \|p - \phi(p)\|_2$, while satisfying, for every nonempty cluster with center $a \in C$ and every color $i \in [c]$,*

$$\alpha_i \leq \frac{|\{p \in P_i : \phi(p) = a\}|}{|\{p \in P : \phi(p) = a\}|} \leq \beta_i.$$

A pair (C, ϕ) satisfying these conditions is called an exactly (α, β)-fair feasible solution.

For our approximation algorithm, we also use the following relaxed fairness notion.

▶ **Definition 2.3** (λ-Relaxed Group Fairness). *For $\lambda > 1$, a clustering is called λ-relaxed (α, β)-fair if, for every nonempty cluster and every color $i \in [c]$,*

$$\frac{\alpha_i}{\lambda} \leq \frac{|\{p \in P_i : \phi(p) = a\}|}{|\{p \in P : \phi(p) = a\}|} \leq \min\{1, \lambda\beta_i\}.$$

Let R denote the minimum radius of an exactly (α, β)-fair solution using at most k continuous centers. Our algorithm returns a clustering of radius at most $(1 + \epsilon)R$ while satisfying the above λ-relaxed fairness condition.

Parameterized Complexity. A parameterized problem has an input of size n together with a parameter k. An algorithm is called fixed-parameter tractable, or FPT, with respect to k if its running time can be written as $f(k)n^{\mathcal{O}(1)}$, where f is an arbitrary computable function depending only on the parameter. More generally, when other quantities such as $1/\varepsilon$ are treated as parameters, the running time may be written as $f(k, 1/\epsilon)n^{\mathcal{O}(1)}$. In contrast, an algorithm with running time $n^{f(k)}$ is said to be in XP with respect to k.

We call the dependence on k subexponential if it is of the form $2^{o(k)}$. For example, for every fixed $d \geq 2$, $2^{\mathcal{O}(k^{1-1/d}\log k)}$ is subexponential in k. An FPT approximation scheme gives, for every $\varepsilon > 0$, a $(1+\varepsilon)$-approximation with an FPT running time in the relevant parameters. The geometric running-time bounds are stated in the standard real-arithmetic model for fixed-dimensional Euclidean algorithms.

Basic Properties. The following three properties will be useful throughout the paper. They capture the basic facts about fairness, count distortion, and minimum enclosing balls that are needed in our algorithm.

Property 1. If non empty clusters $X_1, X_2, \ldots, X_t$ are each (α, β)-fair, then their disjoint union $X = \bigcup_{j=1}^{t} X_j$ is also (α, β)-fair.

Property 2. Let $x_1, x_2, \ldots, x_c \geq 0$ and $x'_1, x'_2, \ldots, x'_c \geq 0$ with totals $N = \sum_i x_i > 0$ and $N' = \sum_i x'_i > 0$. Suppose that for some $\mu \geq 1$, $\frac{x_i}{\mu} \leq x'_i \leq \mu x_i$ holds for every $i \in [c]$. Then,

$$\frac{x_i}{\mu^2 N} \leq \frac{x'_i}{N'} \leq \mu^2 \frac{x_i}{N}$$

Property 3. For every finite nonempty set of points $X \subseteq \mathbb{R}^d$, the radius of minimum enclosing ball of X can be always determined by at most $d+1$ many points [27].

For Property 2, each color proportion in the union is a weighted average of the proportions in the individual clusters. For Property 2, summing the count inequalities gives $N/\mu \leq N' \leq \mu N$, and dividing the bounds for x'_i by these total-size bounds gives the stated inequalities.

3 Technical Overview

Our goal is to form at most k clusters whose points lie close to their centers, while keeping the proportion of each color within its prescribed range. We allow a small increase in radius and a controlled relaxation of fairness. The main idea is to reduce the possible center locations to a finite set, divide the problem into smaller regions, and keep enough information to combine their assignments.

Preprocessing Phase. The algorithm first guesses the radius R. In d-dimensional space, the minimum enclosing ball of any cluster is determined by at most $d+1$ points. Hence, after moving every center to the minimum enclosing ball of its cluster, the fairness does not change, and the radius does not increase. Therefore, it is enough to consider the radii obtained from all subsets of at most $d+1$ input points.

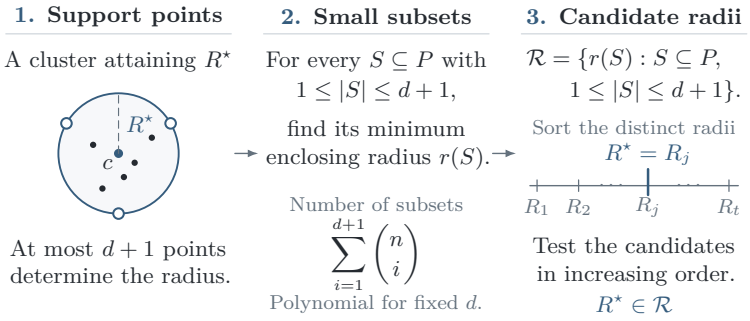


Figure 1 In $\mathbb{R}^d$, a minimum enclosing ball is determined by at most $d+1$ points. Enumerating these small subsets therefore gives a finite candidate set containing the optimal radius $R^\star$.

For a fixed radius R, we build a graph on P by connecting two points whose distance is at most $2R$. Any two points assigned to the same radius-R center must belong to the same connected component. Hence, different connected components can be considered separately. Moreover, if a feasible solution using at most k centers exists, there are at most k such components, and each component has diameter $\mathcal{O}(kR)$. Therefore, each component lies in a bounded region. We enlarge this region slightly and place a fine grid inside it. The grid points are then used as the candidate centers in the remaining algorithm.

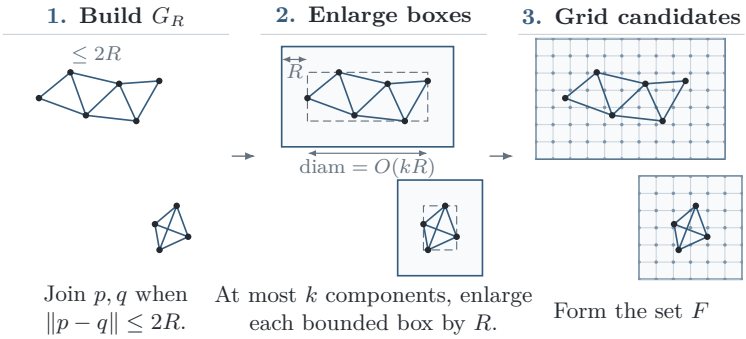


Figure 2 The graph G_R separates the input into connected components. Each component lies in a bounded region, which is enlarged and covered by a fine grid. The grid points are used as candidate centers.

Gridding Phase. Moving a center to a nearby grid point increases the radius only slightly; a sufficiently fine grid keeps it within $(1+\varepsilon)R$. If several centers move to the same location, we merge their clusters (by Property 2, this preserves fairness and uses no more centers). This uses another property: merging fair clusters preserves fairness. The proportion of any color after merging is a weighted average of its proportions in the original clusters, so it remains within the prescribed range.

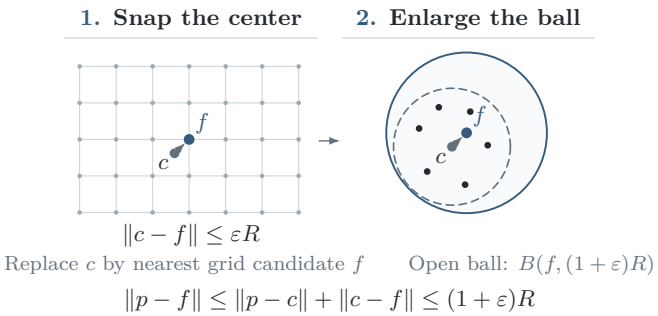


Figure 3 Effect of moving a center to a nearby grid point. The center c moves to its nearest grid point f by a distance of at most εR. Therefore, every point previously covered within radius R remains covered within radius $(1+\varepsilon)R$.

We next divide the problem using a separator sphere. Each candidate center $f \in F$ defines a ball $D_f = B(f, r)$, where $r = R + \varepsilon R/20 < (1+\varepsilon)R$. This ball contains precisely those points that can be assigned to f without exceeding radius r. Since the candidate centers lie on a grid, only a bounded number of these balls can cover the same point. The separator theorem gives a sphere that places a fixed fraction of the selected centers on each

side, while only a small number of their balls cross the sphere. We guess these crossing balls and solve the inside and outside parts recursively.

Sphere-Separator Phase. We next use a geometric separator to divide the problem into smaller parts. For every candidate center $f \in F$, consider the ball $D_f = B(f, r)$, which contains all points at distance at most r from f. A sphere is the boundary of a ball; in two dimensions, it is simply a circle. We use such a sphere as a separator to divide the candidate balls into three groups: those lying completely inside, those lying completely outside, and those crossing the separator.

The main idea behind our subexponential algorithm comes from the sphere-separator theorem. Since the candidate centers are placed on a grid, the family of candidate balls has bounded ply $p = \mathcal{O}_d(\varepsilon^{-d})$. Now consider any ℓ selected candidate balls. The theorem guarantees the existence of a separator sphere γ such that at most $\beta_d \ell$ balls lie completely on either side, where $\beta_d = (d+1)/(d+2) < 1$, and only $\mathcal{O}_d\left(\ell^{1-1/d}/\varepsilon\right)$ balls cross the separator. In particular, for $d = 2$, only $\mathcal{O}(\sqrt{\ell}/\varepsilon)$ balls cross it. Thus, after identifying these crossing balls, we can divide the remaining selected centers into two smaller parts and solve them recursively.

This idea is similar to separator-based dynamic programming on graphs of small treewidth. In such algorithms, a small boundary connects different parts of the graph, and the information stored at this boundary allows the parts to be solved independently. We follow a similar idea, where the crossing balls form the boundary between the two geometric regions. However, our problem is more involved because a center whose ball crosses the separator may receive points of different colors from both sides. Therefore, we must also keep track of the number of points of each color assigned to these centers.

Why a More Careful Recursion Is Needed. Although the separator gives a small boundary, directly applying a standard dynamic programming approach is not sufficient for our required running time. At each step, there may be polynomially many possible separator choices, and the recursion has $\mathcal{O}_d(\log k)$ levels. More importantly, storing every possible color count for the crossing centers would create $n^{\mathcal{O}(ck^{1-1/d}/\varepsilon)}$ states, leading to an XP-type running time rather than an FPT algorithm. To overcome this difficulty, we use a specially designed recursive algorithm. Instead of considering infinitely many separator spheres, we enumerate a finite family of representative spheres that preserve the inside, outside, and crossing classification of every candidate ball. For each representative separator, we guess the selected crossing centers and recursively solve the two sides. We keep these centers active until their complete clusters are known and store their color counts using multiplicative buckets rather than exact values. This allows us to control the number of states and obtain the required subexponential dependence on k. We explain these steps below.

Recursion Phase. With ℓ centers remaining, at most $\mathcal{O}_d(\ell^{1-1/d}/\epsilon)$ selected balls need to cross. We enumerate the relevant separators and guess the selected crossing balls before solving the two sides recursively. A crossing cluster may receive differently colored points from both sides, whose union is fair. We therefore keep it active, record the color counts contributed by each side, and check fairness only after all contributions have been collected. A direct use of this idea gives a $k^{\mathcal{O}(k)}$-type algorithm; obtaining a subexponential running time requires the more careful recursion described below.

Bucketing Subroutine. Storing every possible color count would make the number of states too large and lead to an XP-type algorithm. We therefore use bucketing: zero is stored separately, while nearby positive counts are grouped into the same geometric interval. Thus, the algorithm remembers only an approximate value of each count instead of its

exact value, which greatly reduces the number of choices. For one center-color pair, we use $L = \mathcal{O}(1 + \log(k+1)\log(n+1)/\log\lambda)$ buckets. Along a recursive path there are at most $H_d = \mathcal{O}_d(k^{1-1/d}/\varepsilon)$ active centers, so a table has at most L^{cH_d} count states. The factor involving $\log n$ can be absorbed into $n^{\mathcal{O}(1)}$ together with a parameter-dependent factor, as shown in Section A.3.

This approximation is justified by Property 2. If every color count changes by at most a factor μ, then the total cluster size also changes by at most μ. Therefore, every color proportion changes by at most a factor μ^2. Hence, using sufficiently fine buckets introduces only a controlled error in fairness. This bucketing step reduces the state space and helps obtain a subexponential FPT algorithm instead of an XP algorithm. Adding approximate counts does not increase the error factor. Error is added only when the counts are rounded again. Since the recursion has $\mathcal{O}(\log k)$ levels, we choose narrow buckets so that the total error remains within the allowed fairness relaxation.

Flowing Subroutine. Finally, we use a lower and upper bounded feasible flow for each color to construct an assignment consistent with the chosen bucket intervals. Every point is assigned to an eligible center, and the number of points of each color assigned to a center remains within its stored interval. Since all capacities are integral, a feasible flow gives an integral assignment. Once a cluster is complete, we check its actual color counts against the relaxed fairness bounds. The small number of active centers and the compact representation of their color counts give the required subexponential dependence on k, for fixed dimension, number of colors, and approximation parameters.

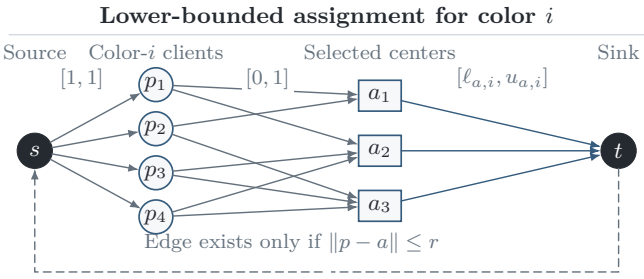


Figure 4 Lower-bounded feasible flow for one color i. Every client sends exactly one unit of flow to an eligible center. The bounds on the center-to-sink edges ensure that the number of color-i clients assigned to each center lies in its selected bucket interval. The same construction is applied independently to every color.

Subexponential Time Bound. The main saving in the running time comes from the separator. A direct approach would guess all k centers from the grid and would require roughly $(k/\epsilon)^{\mathcal{O}(k)}$ choices. In our recursion, however, only the centers whose balls cross the separator need to be guessed and kept active. Their total number along a recursive branch is $\mathcal{O}_d\left(\frac{k^{1-1/d}}{\varepsilon}\right)$. The recursion has $\mathcal{O}_d(\log k)$ levels, while the separator choices, rounded color counts, and flow computations contribute only additional parameter-dependent or polynomial factors. Therefore, for fixed dimension and number of colors, the overall running time is $n^{\mathcal{O}(d)}\left(\frac{k}{\epsilon}\right)^{\mathcal{O}_{c,d,\lambda}(k^{1-1/d}/\epsilon)}$. Since $k^{1-1/d} = o(k)$, the dependence on k is subexponential rather than exponential. A detailed analysis of the running time is provided in Section A.3.

The complete algorithm, its correctness proof, and the running-time analysis are provided in the appendix A.

4 Conclusion

We studied the group -fair k-center problem in fixed dimension, where the input points are divided into disjoint classes, and every cluster must satisfy given fairness conditions. By combining grid-based discretization, a sphere-separator method, and a recursive algorithm, we obtain a $(1+\epsilon)$- approximation for the radius while allowing a multiplicative λ -relaxation of the fairness constraints. The main idea is to use a separator to divide the problem into smaller parts. During the recursion, we keep only a small number of active centers and store their color counts using multiplicative buckets. This reduces the number of recursion states while still preserving fairness within the allowed relaxation.

Compared with earlier work on fair clustering in more general metric spaces, we studied the problem on Euclidean space to obtain better dependence on k. As a direction for further study, it would be interesting to remove the fairness relaxation, handle overlapping color groups, and study whether a similar subexponential approach can be extended to more general geometric settings.

5 AI-Assisted Preparation

The initial research ideas and mathematical intuition originated with the authors. Through interactive discussions, OpenAI's ChatGPT 6.1 Sol assisted in formalizing these ideas, addressing gaps in the arguments, and exploring possible algorithmic improvements. The manuscript was composed by the authors, with AI assistance in refining selected passages. All mathematical arguments and algorithmic claims were independently verified by the authors, who take full responsibility for the final content.

References

1 Pankaj K. Agarwal and Cecilia Magdalena Procopiuc. Exact and approximation algorithms for clustering. *Algorithmica*, 33(2):201–226, 2002. URL: `https://doi.org/10.1007/s00453-001-0110-y`, `doi:10.1007/S00453-001-0110-Y`.

2 Gagan Aggarwal, Rina Panigrahy, Tomás Feder, Dilys Thomas, Krishnaram Kenthapadi, Samir Khuller, and An Zhu. Achieving anonymity via clustering. *ACM Trans. Algorithms*, 6(3):49:1–49:19, 2010. `doi:10.1145/1798596.1798602`.

3 Sara Ahmadian, Alessandro Epasto, Ravi Kumar, and Mohammad Mahdian. Clustering without over-representation. In Ankur Teredesai, Vipin Kumar, Ying Li, Rómer Rosales, Evimaria Terzi, and George Karypis, editors, *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2019, Anchorage, AK, USA, August 4-8, 2019*, pages 267–275. ACM, 2019. `doi:10.1145/3292500.3330987`.

4 Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. *Network flows - theory, algorithms and applications*. Prentice Hall, 1993.

5 Sayan Bandyapadhyay, Fedor V. Fomin, and Kirill Simonov. On coresets for fair clustering in metric and euclidean spaces and their applications. In Nikhil Bansal, Emanuela Merelli, and James Worrell, editors, *48th International Colloquium on Automata, Languages, and Programming, ICALP 2021, Glasgow, Scotland (Virtual Conference), July 12-16, 2021*, volume 198 of *LIPIcs*, pages 23:1–23:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021. URL: `https://doi.org/10.4230/LIPIcs.ICALP.2021.23`, `doi:10.4230/LIPICS.ICALP.2021.23`.

6 Saugata Basu, Richard Pollack, and Marie-Françoise Roy. *Algorithms in real algebraic geometry*. Springer, 2006.

7 Suman Kalyan Bera, Deeparnab Chakrabarty, Nicolas Flores, and Maryam Negahbani. Fair algorithms for clustering. In Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett, editors, *Advances*

in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada, pages 4955–4966, 2019. URL: https://proceedings.neurips.cc/paper/2019/hash/fc192b0c0d270dbf41870a63a8c76c2f-Abstract.html.

8 Ioana Oriana Bercea, Martin Groß, Samir Khuller, Aounon Kumar, Clemens Rösner, Daniel R. Schmidt, and Melanie Schmidt. On the cost of essentially fair clusterings. In Dimitris Achlioptas and László A. Végh, editors, *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques, APPROX/RANDOM 2019, Massachusetts Institute of Technology, Cambridge, MA, USA, September 20-22, 2019*, volume 145 of *LIPIcs*, pages 18:1–18:22. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019. URL: https://doi.org/10.4230/LIPIcs.APPROX-RANDOM.2019.18, doi:10.4230/LIPICS.APPROX-RANDOM.2019.18.

9 Matteo Ceccarello, Andrea Pietracaprina, and Geppino Pucci. Fast and accurate fair k-center clustering in doubling metrics. In Tat-Seng Chua, Chong-Wah Ngo, Ravi Kumar, Hady W. Lauw, and Roy Ka-Wei Lee, editors, *Proceedings of the ACM on Web Conference 2024, WWW 2024, Singapore, May 13-17, 2024*, pages 756–767. ACM, 2024. doi:10.1145/3589334.3645568.

10 Moses Charikar, Samir Khuller, David M. Mount, and Giri Narasimhan. Algorithms for facility location problems with outliers. In S. Rao Kosaraju, editor, *Proceedings of the Twelfth Annual Symposium on Discrete Algorithms, January 7-9, 2001, Washington, DC, USA*, pages 642–651. ACM/SIAM, 2001. URL: http://dl.acm.org/citation.cfm?id=365411.365555.

11 Xingyu Chen, Brandon Fain, Liang Lyu, and Kamesh Munagala. Proportionally fair clustering. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, volume 97 of *Proceedings of Machine Learning Research*, pages 1032–1041. PMLR, 2019. URL: http://proceedings.mlr.press/v97/chen19d.html.

12 Flavio Chierichetti, Ravi Kumar, Silvio Lattanzi, and Sergei Vassilvitskii. Fair clustering through fairlets. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pages 5029–5037, 2017. URL: https://proceedings.neurips.cc/paper/2017/hash/978fce5bcc4eccc88ad48ce3914124a2-Abstract.html.

13 Rajesh Chitnis and Nitin Saurabh. Tight lower bounds for approximate & exact k-center in ℓ_∞^d. In Xavier Goaoc and Michael Kerber, editors, *38th International Symposium on Computational Geometry, SoCG 2022, Berlin, Germany, June 7-10, 2022*, volume 224 of *LIPIcs*, pages 28:1–28:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022. URL: https://doi.org/10.4230/LIPIcs.SoCG.2022.28, doi:10.4230/LIPICS.SOCG.2022.28.

14 Marek Cygan, MohammadTaghi Hajiaghayi, and Samir Khuller. LP rounding for k-centers with non-uniform hard capacities. In *53rd Annual IEEE Symposium on Foundations of Computer Science, FOCS 2012, New Brunswick, NJ, USA, October 20-23, 2012*, pages 273–282. IEEE Computer Society, 2012. doi:10.1109/FOCS.2012.63.

15 Erik D. Demaine, Fedor V. Fomin, Mohammad Taghi Hajiaghayi, and Dimitrios M. Thilikos. Fixed-parameter algorithms for (k, r)-center in planar graphs and map graphs. *ACM Trans. Algorithms*, 1(1):33–47, 2005. doi:10.1145/1077464.1077468.

16 Seyed A. Esmaeili, Brian Brubach, Aravind Srinivasan, and John Dickerson. Fair clustering under a bounded cost. In Marc'Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan, editors, *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pages 14345–14357, 2021. URL: https://proceedings.neurips.cc/paper/2021/hash/781877bda0783aac5f1cf765c128b437-Abstract.html.

17 Teofilo F. Gonzalez. Clustering to minimize the maximum intercluster distance. *Theor. Comput. Sci.*, 38:293–306, 1985. doi:10.1016/0304-3975(85)90224-5.

18 Dishant Goyal and Ragesh Jaiswal. Tight FPT approximation for constrained k-center and k-supplier. *Theor. Comput. Sci.*, 940(Part):190–208, 2023. URL: https://doi.org/10.1016/j.tcs.2022.11.001, doi:10.1016/J.TCS.2022.11.001.

19 Elfarouk Harb and Ho Shan Lam. KFC: A scalable approximation algorithm for k-center fair clustering. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. URL: https://proceedings.neurips.cc/paper/2020/hash/a6d259bfbfa2062843ef543e21d7ec8e-Abstract.html.

20 Dorit S. Hochbaum and David B. Shmoys. A best possible heuristic for the k-center problem. *Math. Oper. Res.*, 10(2):180–184, 1985. URL: https://doi.org/10.1287/moor.10.2.180, doi:10.1287/MOOR.10.2.180.

21 Ting Liang, Xiaoliang Wu, Jinhui Xu, and Qilong Feng. The distributed algorithms for the lower-bounded k-center clustering in metric space. *Theor. Comput. Sci.*, 1027:114975, 2025. URL: https://doi.org/10.1016/j.tcs.2024.114975, doi:10.1016/J.TCS.2024.114975.

22 Sepideh Mahabadi and Ali Vakilian. Individual fairness for k-clustering. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, pages 6586–6596. PMLR, 2020. URL: http://proceedings.mlr.press/v119/mahabadi20a.html.

23 Evi Micha and Nisarg Shah. Proportionally fair clustering revisited. In Artur Czumaj, Anuj Dawar, and Emanuela Merelli, editors, *47th International Colloquium on Automata, Languages, and Programming, ICALP 2020, Saarbrücken, Germany (Virtual Conference), July 8-11, 2020*, volume 168 of *LIPIcs*, pages 85:1–85:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020. URL: https://doi.org/10.4230/LIPIcs.ICALP.2020.85, doi:10.4230/LIPICS.ICALP.2020.85.

24 Gary L. Miller, Shang-Hua Teng, William P. Thurston, and Stephen A. Vavasis. Separators for sphere-packings and nearest neighbor graphs. *J. ACM*, 44(1):1–29, 1997. doi:10.1145/256292.256294.

25 Ayano Moritaka, Shin-ichi Nakano, Kento Tanaka, and Noriaki Yoshida. Euclidean k-center fair clusterings. *CoRR*, abs/2511.07660, 2025. URL: https://doi.org/10.48550/arXiv.2511.07660, arXiv:2511.07660, doi:10.48550/ARXIV.2511.07660.

26 Maryam Negahbani and Deeparnab Chakrabarty. Better algorithms for individually fair k-clustering. In Marc'Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan, editors, *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pages 13340–13351, 2021. URL: https://proceedings.neurips.cc/paper/2021/hash/6f221fcb5c504fe96789df252123770b-Abstract.html.

27 Emo Welzl. Smallest enclosing disks (balls and ellipsoids). In Hermann A. Maurer, editor, *New Results and New Trends in Computer Science, Graz, Austria, June 20-21, 1991, Proceedings [on occasion of H. Maurer's 50th birthday]*, volume 555 of *Lecture Notes in Computer Science*, pages 359–370. Springer, 1991. URL: https://doi.org/10.1007/BFb0038202, doi:10.1007/BFB0038202.

28 Xiaoliang Wu, Qilong Feng, Ziyun Huang, Jinhui Xu, and Jianxin Wang. New algorithms for distributed fair k-center clustering: Almost accurate as sequential algorithms. In Mehdi Dastani, Jaime Simão Sichman, Natasha Alechina, and Virginia Dignum, editors, *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems, AAMAS 2024, Auckland, New Zealand, May 6-10, 2024*, pages 1938–1946. International Foundation for Autonomous Agents and Multiagent Systems / ACM, 2024. URL: https://dl.acm.org/doi/10.5555/3635637.3663057, doi:10.5555/3635637.3663057.

29 Xiaoliang Wu, Ting Liang, Zhize Li, and Qilong Feng. Parameterized approximation schemes for fair clustering in doubling metrics. In Diego Calvanese, editor, *Proceedings of the Thirty-Fifth International Joint Conference on Artificial Intelligence, IJCAI-26*, pages 817–825.

International Joint Conferences on Artificial Intelligence Organization, 8 2026. Main Track. doi:10.24963/ijcai.2026/92.

A Algorithm for Group-Fair Euclidean k-Center

A.1 Algorithm

We first describe the radius guessing and grid preprocessing steps that produce a finite set of candidate centers. We then present the separator recursion, color-count bucketing, and flow-based assignment subroutine, followed by the correctness proof and running-time analysis.

Guess 1: In starting of our algorithm, we want to guess the radius of the optimal clustering C. Suppose R^* is the optimal radius. In an optimal clustering C, consider the cluster whose minimum enclosing ball has radius R^*. Based on property 2, we can say, R^* is the radius of the minimum enclosing ball determined by some subset of at most $d+1$ input points. Hence we enumerate all possible on all such subset and compute their minimum enclosing ball radius. This gives at most $\mathcal{O}(n^{d+1})$ many candidate radius. For a fixed dimension, this can be done in poly(n) time. So, our guess R tries all possible values in increasing order.

The value $R = 0$ is handled separately: clients at each distinct location must form a fair cluster, after merging all clusters at that location. We return these clusters if there are at most k locations and all of them satisfy the exact fairness bounds; otherwise we continue with the positive candidate radii. This avoids using a grid of spacing zero.

Construction of the Graph G_R: After fixing a candidate guess $R > 0$, we construct a graph G_R from the given set of points P where each vertex of G_R corresponds a point of P and we put an edge between two vertex p, q if $d(p, q) = \|p - q\|_2 \leq 2R$ in $\mathbb{R}^d$. Hence come our two observations-

▶ **Observation A.1.** *Every cluster of every radius R-solution lies in a single connected component of G_R.*

Proof. Consider any two points p, q from a same cluster X whose center is say a. Then, according to the construction of our graph G_R, $\text{dist}(p, q) = \|p-q\|_2 \leq \|p-a\|_2+\|a-q\|_2 \leq 2R$. Which implies that every cluster X induces a clique in G_R. ◀

▶ **Observation A.2.** *If G_R has more than k connected components, then no radius R solution clustering of at most k centers exists.*

Hence we discard this choice and move on to the next guess.

▶ **Observation A.3.** *Suppose a connected component H of G_R is covered by $m(\leq k)$ radius R clusters. Then, every side of its axis-parallel bounding box is $\mathcal{O}(kR)$.*

Proof. Let the m centers contanied in H is $a_1, a_2, \ldots, a_m$. Now suppose there is an edge $pq \in E(G_R)$ such that, center of p is a_i and center of q is a_j for some $i \neq j \in [m]$. By triangle inequality, $\|a_i - a_j\| \leq \|a_i - p\|_2 + \|p - q\|_2 + \|q - a_j\|_2 \leq 4R$. Since, H has m such centers, any two centers can be connected through a path of at most $m - 1$ such edges. Therefore their distance can be at most $4R(m - 1)$. Finally, for any two arbitrary clients x, y the distance can be at most $(4m - 2)R$. So, the diameter of the component can be at most $(4m - 2)R \leq (4k - 2)R$. Length of each side of the bounding box is $\mathcal{O}(kR)$. ◀

If any component violates this diameter bound, reject the guess R.

Griding the Whole Instance: For each component H of G_R, consider its bounding box enlarged by R in every direction. As every center to which a point of H can be assigned will lie within distance R of some points of H. We therefore place a grid of spacing $\delta = \frac{\epsilon R}{10\sqrt{d}}$

only inside these bounded regions and take the resulting grid points as candidate centers. We denote F_H is the set of grid points for the component H and $F = \bigcup_H F_H$.

All boxes use the same grid $\delta\mathbb{Z}^d$. We extend each enlarged box outward to the nearest grid hyperplanes and take all grid points in this extended box, removing duplicate locations in F. Thus, all distinct candidate centers are δ-separated, even if boxes overlap, and the nearest grid point to every relevant radius-R center is included.

▶ **Lemma A.4.** *For every fixed R, the total number of grid points are $|F| = \mathcal{O}\left(\frac{k^{d+1}d^{d/2}}{\epsilon^d}\right)$.*

Proof. Side length of each bounding box is $\mathcal{O}(kR)$. So in one coordinate direction, the number of grid cell is $\mathcal{O}\left(\frac{k\sqrt{d}}{\epsilon}\right)$. There are total at most k components over d coordinates. Hence, total number of grid cells are, $\mathcal{O}\left(k\left(\frac{k\sqrt{d}}{\epsilon}\right)^d\right) = \mathcal{O}\left(\frac{k^{d+1}d^{d/2}}{\epsilon^d}\right)$. ◀

Also, notice that, any radius-R center serving a nonempty cluster are distance at most $\delta\sqrt{d}/2$ from a grid point of F. Define, $r = R + \frac{\delta\sqrt{d}}{2} = R + \frac{\epsilon R}{20} < (1+\epsilon)R$.

▶ **Lemma A.5.** *If an exactly (α, β)-fair radius R solution exists with k centers, then there is an exactly (α, β)-fair solution of radius at most r whose centers all lie in F and which uses at most k centers.*

Proof. Suppose there is an exactly (α, β)-fair solution of optimal radius R of k center clustering. Consider an arbitrary center a of this solution and map it to the nearest grid point $f \in F$. Clearly, the distance between a and f is at most $\delta\sqrt{d}/2$. Now suppose a client p is assigned to a in the actual solution. Then, after moving the center to f, $||p - f|| \leq ||p - a|| + ||a - f|| \leq r$. Hence, considering all grid points as the candidate center can increase the radius at most $r \leq (1+\epsilon)R$. If several centers map to the same grid point, merge their clusters. Property 2 preserves exact fairness, and the number of centers cannot increase. ◀

Now, for each $f \in F$, consider a closed ball $D_f = B(f, r)$ of radius r. It consists of all clients that may be assigned to f without exceeding radius r.

▶ **Definition A.6.** *A family of balls is called p-ply if every point of that space can belong to at most p balls. Formally, for a family of balls $\{D_f : f \in F\}$, its ply is $p = \max_{x\in\mathbb{R}^d} |\{f \in F : x \in D_f\}|$.*

▶ **Lemma A.7.** *The family $\{D_f : f \in F\}$ has ply $p = \mathcal{O}((\sqrt{d}/\epsilon)^d)$.*

Proof. Consider an arbitrary point $x \in \mathbb{R}^d$. We want to count total number of balls $D_f = B(f, r)$ that can contain x. If $x \in D_f$, then by definition, $\|f - x\| \leq r$. So, every point f whose closed ball contains x must lie inside $B(x, r)$. Hence, it is sufficient to count then umber of grid points that lie in $B(x, r)$. Each grid point is δ-separated. So, if we draw a ball $B(f, \delta/2)$ of radius $\delta/2$, then their interiors are disjoint. Also, each f lies within distance r of x, so all these small balls also lie inside $B(x, \delta/2 + r)$, with a slightly larger radius. Now suppose there ar N such candidate centers which contains x. Then, $N \cdot \text{vol}\left(B\left(0, \frac{\delta}{2}\right)\right) \leq \text{vol}\left(B\left(0, r + \frac{\delta}{2}\right)\right)$. Since the volume of a d-dimensional ball of radius t is proportional to t^d, $N \leq \left(\frac{r+\delta/2}{\delta/2}\right)^d = \left(\frac{2r}{\delta} + 1\right)^d$. Now $\delta = \frac{\varepsilon R}{10\sqrt{d}}$. Therefore, $\frac{r}{\delta} = \mathcal{O}\left(\frac{\sqrt{d}}{\varepsilon}\right)$. Hence, $N = \mathcal{O}\left(\left(\frac{\sqrt{d}}{\varepsilon} + 1\right)^d\right)$. Thus, every point x belongs to at most $\mathcal{O}_d(\epsilon^{-d})$ candidate balls. ◀

Existence of Sphere Separator: Here, we can use the sphere separator theorem for the family of closed balls with a bounded ply [24].

▶ **Theorem A.8** (Sphere separator). *Let Γ be a p-ply family of a m balls in $\mathbb{R}^d$. There exists a $(d-1)$-dimensional sphere γ such that,*

- *At most $\mathcal{O}(C_d p^{1/d} m^{1-1/d})$ balls that intersects γ, where C_d is a constant dependent on dimention d.*
- *At most $\beta_d m$ balls lie completely inside γ, where $\beta_d = \frac{d+1}{d+2} < 1$.*
- *At most $\beta_d m$ balls lie completely outside γ.*

Hence by lemma A.7 and theorem A.8 at each step applied to any set of at most ℓ selected candidate balls, there is a separator with $w_d(\ell) = \min\{\ell, \lceil C_d p^{1/d} \ell^{1-1/d} \rceil\} = \mathcal{O}_d(\ell^{1-1/d}/\varepsilon)$ selected balls which crosses the separator.

Finding the Separator: So, from the previous theorem, we know the existence of such a separator, but if we want to find such sphere separator, it is not possible to enumerate all such infinitely many possibilities. Hemce, we want to reduce the range of enumeration into finitely many choices. Consider a sphere separator in $\mathbb{R}^d$ specified as $(x_1, x_2, \ldots, x_d, \rho)$, where $\rho \geq 0$ is the radius of that sphere separator. Now, a fix candidate ball $D_f = B(f, r)$ and le $h = ||x - f||_2$. Notice that, the candidate ball D_f has only three status relative to the separator :

1. $\rho - r > 0$ and $h^2 < (\rho - r)^2$ iff the ball D_f is completely inside the separator.
2. $h^2 > (\rho + r)^2$ iff D_f lies completely outside the separator.
3. Neither of the above holds iff the ball is crossing or touching the separator.

Hence, for each f we introduce three constant degree polynomials,

$$g_f^-(x, \rho) = \|x - f\|_2^2 - (\rho - r)^2,$$

$$g_f^+(x, \rho) = \|x - f\|_2^2 - (\rho + r)^2,$$

$$h_f(x, \rho) = \rho - r.$$

For each candidate ball D_f, the signs of g_f^-, g_f^+ and h_f completely determine whether D_f lies inside the separator, outside or intersects it. Since there are $M = |F|$ many candidate balls, we obtain total $3M + 1$ polynomials in the $d + 1$ parameters $(x_1, x_2, \ldots, x_d, \rho)$. We also include the polynomial ρ to restrict to nonnegative radii. Each of these polynomials has constant degree. All these polynomials divide the space into $M^{\mathcal{O}(d)}$ regions in a way that, inside one range, the sign of every polynomial remains unchanged. Hence, every separator sphere is treated as equivalent, as it gives exactly the same inside, outside, or crossing classification of all the candidate balls.

We therefore choose one representative point from every non-empty region with $\rho \geq 0$. This includes sign conditions in which some polynomials are zero, so separators touching candidate balls are not lost. Standard algorithms for realizable sign conditions compute these sample points in $M^{\mathcal{O}(d)}$ arithmetic operations for fixed d [6, Chapter 13]. The selected sphere represents all separator spheres in that region.

Hence, we can sum up the whole concept and claim that,

▷ Claim A.9. A list of $M^{\mathcal{O}(d)}$ representative spheres can be computed such that, for every sphere γ, the list contains a sphere separator γ^* with the same inside, outside, and crossing classification for every candidate ball.

Solving Three Parts Recursively: In the previous section, we showed that a separator sphere divides the selected candidate balls into three groups. So, the balls lying completely

inside and completely outside can be handled separately. So, we can solve these two cases through recursion. However, a selected ball that crosses the separator may have assigned clients from both sides. So, the cluster formed by this center can not be completed immediately and has to remain active during the solution of all its subproblems. After choosing a separator, we define a recursive state as $sol(P', F', A, l)$ where,

- P' = the set of clients that belong to the current region handled by this current recursive call. For the root, i.e., the 1st recursive call, $P = P'$.
- F' = all the candidate centers whose selection is not yet decided and closed balls of each center lies completely within that current region. At the start, $F = F'$.
- A = The set of centers that have already been selected but whose clusters are not yet complete. We call them active centers.
- l = The number of additional centers that may still be selected from F'.

For a fixed call, the table is indexed only by the bucket vector of the inherited active centers in A. Each entry keeps one integral witness: its selected centers, the assignment of all clients in P', and the exact color counts of that assignment. Every nonempty cluster at a newly selected center is complete within this region and must pass the relaxed fairness test. Inherited active centers are not tested yet, since their contributions from other regions are still missing. If several witnesses have the same bucket vector, keep any one of them. The clients and candidate centers in a child call are always restricted to the current region. For every active center $a \in A$, the current recursive call also keeps track of how many clients of color i are assigned to a from the current region. We write this as a vector for each center a, $x_a = (x_{a,1}, x_{a,2}, \ldots, x_{a,c})$, where, $x_{a,i}$ denotes the number of color i clients assigned to a from this region. When different recursive parts are later combined, their contribution to the same active center are added together. But if a state wants to keep the exact counts and enumerate on all possible values for $x_{a,i}$, it will result in a lot of states. So, to keep the number of states small, the algorithm stores a rounded version of the counts.

Rounding the assigned points count: For an active center $a \in A$, the DP will store the number of clients of each color that have already been assigned to a. These numbers can be any integers between 0 and n. To decrease the number of DP states, we group the nearby counts into multiplicative buckets and store only the corresponding bucket indices. An active center may remain active through several recursive levels. Hence, the count will be updated every time before the center is finally completed. We first bound how many such recursive levels are possible. Let $D_d = \left\lceil \log_{1/\beta_d} k \right\rceil + 1 = \mathcal{O}_d(\log(k+1))$, where $\beta_d = \frac{d+1}{d+2} < 1$. At a recursive node with remaining center ℓ, each child has at most $\beta_d \ell$. Thus, along any root-to-leaf recursion path, the number of undecided centers decreases as $k,\ \beta_d k,\ \beta_d^2 k, \ldots$ until the value become at most 1. Therefore, D_d is a safe upper bound on the recursion depth.

Now we choose the width of the buckets. Let $\theta = \lambda^{1/(4(D_d+2))} > 1$. The value of θ is chosen very close to 1 as the same count may be rounded at different recursive levels. One rounding step will introduce at most a factor-θ change. There are at most $D_d + 2$ relevant distortion steps along a recursive path, and adding contributions from different paths does not increase the error factor. Hence the total error can be at most $\theta^{D_d+2} = \lambda^{1/4}$.

For a positive integer x, define $\text{bucket}(x) = \lfloor \log_\theta x \rfloor$. If $b = \text{bucket}(x)$, then $\theta^b \le x < \theta^{b+1}$. Hence, bucket b represents the integer interval $I_b = \left\{ z \in \mathbb{Z}_{>0} : \theta^b \le z < \theta^{b+1} \right\}$. A special zero bucket represents the value 0. This is necessary because multiplicative rounding cannot represent zero, and a zero color count must remain exactly zero. Since every color count is at most n, the number of possible buckets for one center-color pair is at most $L = 2 + \lfloor \log_\theta n \rfloor$.

By the definition of θ, $\log_\theta n = \frac{\log n}{\log \theta} = \frac{4(D_d+2)\log n}{\log \lambda}$. Therefore, $L = \mathcal{O}\left(1 + \frac{D_d \log n}{\log \lambda}\right)$. Buckets whose intervals contain no integer between 1 and n are simply ignored. The following simple observation shows why this rounding is useful.

▶ **Observation A.10.** *If x and y are positive integers in the same bucket, then $\frac{x}{\theta} < y < \theta x$.*

Proof. If $x, y > 0$ and are in the same bucket I_b, then $\theta^b \le x, y < \theta^{b+1}$. Since, $x < \theta^{b+1}$ and $y \ge \theta^b$ we get $x < \theta y$. Thus, $y > \frac{x}{\theta}$. Similarly, by exchanging the roles of x and y, we have $y < \theta x$. Therefore, $\frac{x}{\theta} < y < \theta x$. Hence, replacing one witness count by another count from the same bucket changes the count by less than a multiplicative factor of θ. ◀

▶ **Observation A.11.** *Let $z_1, z_2, \ldots, z_t \ge 0$ and $z'_1, z'_2, \ldots, z'_t \ge 0$. If for all $j \in [t]$ and for some $\eta \ge 1$ $\frac{z_j}{\eta} \le z'_j \le \eta z_j$, then*

$$\sum_j \frac{z_j}{\eta} \le \sum_j z'_j \le \eta \sum_j z_j.$$

▶ **Lemma A.12.** *Suppose a nonempty cluster is exactly (α, β)-fair in a reference assignment. Let $x_1, x_2, \ldots, x_c$ be its exact color counts, and let $x'_1, x'_2, \ldots, x'_c$ be the actual counts of another integral cluster. Assume that $x_i/\eta \le x'_i \le \eta x_i$ for every $i \in [c]$, where $\eta = \theta^{D_d+2} \le \lambda^{1/4}$. These bounds preserve zero counts and imply that the new cluster is nonempty. Consequently,*

$$\frac{\alpha_i}{\lambda} \le \frac{x'_i}{\sum_j x'_j} \le \min\{1, \lambda\beta_i\} \text{ for every color } i \in [c].$$

Proof. Set $\mu = \eta = \theta^{D_d+2} \le \lambda^{1/4}$. The count-distortion bound is an assumption of this lemma; Lemma A.17 proves it for the witness following the reference assignment. By Property 2, changing a factor-μ in all color counts changes every color proportion by at most a factor of μ^2. Since $\mu^2 \le \lambda^{1/2} \le \lambda$, an original proportion in $[\alpha_i, \beta_i]$ becomes a proportion in $\left[\frac{\alpha_i}{\lambda}, \lambda\beta_i\right]$. Since a proportion is at most 1, the upper bound is $\min\{1, \lambda\beta_i\}$. Hence the result follows. ◀

Assigning the points through flow concept: Now we show how the algorithm obtains an actual assignment of clients once the bucket intervals are fixed. We do not solve the whole fair-assignment problem directly by flow. Instead, the DP first fixes a bucket for every center-color pair, and the flow only checks whether these bucket ranges can be realized by an assignment. Let $Y \subseteq P$ be the set of clients that must be assigned to a fixed set of selected centers A'. For every $a \in A'$ and color i, suppose a bucket $b_{a,i}$ has been fixed. Let $\ell_{a,i} = \min I_{b_{a,i}}$ and $u_{a,i} = \max\left(I_{b_{a,i}} \cap \{0, 1, \ldots, n\}\right)$. Thus, center a must receive between $\ell_{a,i}$ and $u_{a,i}$ clients of color i. For the zero bucket, both values are 0. For every color i, construct the following flow network:

- add an edge from the source s to every client $p \in Y \cap P_i$ with capacity $[1, 1]$;
- add an edge from a client p to a center $a \in A'$ whenever $\|p - a\|_2 \le r$, with capacity $[0, 1]$;
- add an edge from every center a to the sink t with lower and upper capacities $[\ell_{a,i}, u_{a,i}]$.

We add an edge from t to s with lower and upper capacities both equal to $|Y \cap P_i|$, and ask for a feasible circulation. Subtracting the lower bounds and adding a super-source and a super-sink reduces this to a standard maximum-flow feasibility test [4]. No costs are needed. The first type of edge ensures that every client is assigned exactly once. The second type allows a client to be assigned only to a center within distance r. The last type ensures that the number of color-i clients assigned to each center lies in its fixed bucket

interval. Since every client belongs to exactly one color class and there is no capacity shared between different colors at a center, the c color classes can be handled by c independent flow computations.

▶ **Lemma A.13.** *For fixed bucket intervals, feasibility of assigning Y to A' can be decided by c independent polynomial-time flow computations. If a feasible assignment exists, an integral assignment can be obtained. Moreover, if there is a valid integral assignment of Y to A', then one of the bucket vectors enumerated by the algorithm gives a feasible flow instance.*

Proof. For a fixed color i, every client sends exactly one unit of flow and can send it only to a center within distance r. The lower and upper capacities on the center-to-sink edges ensure that each center receives a number of color-i clients lying in its fixed bucket interval. The color networks are independent because the color classes form a partition of the clients and there is no shared capacity between different colors. All capacities and lower bounds are integral. Hence, whenever the flow instance is feasible, an integral feasible flow exists, which directly gives an assignment of the clients. Now suppose a valid integral assignment already exists. For every center a and color i, consider the exact number of color-i clients assigned to a, and choose the bucket containing this number. Since the algorithm enumerates all bucket vectors, it eventually considers this choice. The given assignment itself satisfies all the corresponding lower and upper bounds, so the resulting flow instance is feasible. ◀

How one recursive call works : After the previous steps, we now have all the information needed for the recursion. We next explain how one recursive step splits the current problem into inside and outside parts and handles the centers that cross the separator. Consider a call, $\mathrm{sol}(P', F', A, \ell)$ with $\ell > 1$. For every fix representative separator sphere γ, let $X_\gamma = \{f \in F' : D_f \text{ crosses } \gamma\}$. The algorithm guesses which of these candidate centers (crossings γ) are actually selected. It therefore again enumerates every set $Q \subset X_\gamma$ with $|Q| \leq w_d(\ell)$. Hence, the candidate centers in Q are the selected centers, while all the candidate centers in $X_\gamma \setminus Q$ are rejected in this branch. We then set $A' = A \cup Q$. The centers in A' are active because they may receive client points from both sides of the separator. Now come the non-intersecting parts- let F_{in} and F_{out} be the non-crossing candidate centers whose balls lie completely inside and completely outside of γ. Similarly, P_{in}, P_{out}, and P_{cross} are the client points that lie inside, outside, and on the separator, respectively. The algorithm then guesses the possible number of centers ℓ_{in}, ℓ_{out} from both parts such that $\ell_{in}, \ell_{out} \leq \beta_d \ell$ and $|Q| + \ell_{in} + \ell_{out} \leq \ell$. The sets $P_{in}, P_{out}, P_{cross}$ are a partition of P', and both F_{in} and F_{out} are subsets of F'. A representative separator need not place clients on the same sides as the original separator: preserving the candidate-ball classifications is sufficient, since every client served by a noncrossing ball still lies on the same side as that ball. All other assigned clients can use the selected crossing centers.

Here, it recursively solve $\mathrm{sol}(P_{in}, F_{in}, A', \ell_{in})$ and $sol(P_{out}, F_{out}, A', \ell_{out})$. After both recursive calls return, their contribution to every center are combined. The boundary clients in P_{cross} are then assigned to the active centers using the bucketing concept and finding a feasible flow as described in the previous steps. A ball lying completely inside γ can not contain an outside or boundary client point. A ball completely lying outside γ can not contain an inside or boundary client point. Consequently, after the selected crossing set Q is guessed, every interaction between two recursive sides goes through only active centers in $A' = A \cup Q$.

When combining states, add their exact witness counts, rather than bucket endpoints. A compatible pair consists of two witnesses for these disjoint client sets and the same active-center set A'; the inside and outside counts need not be equal. For each boundary bucket

vector, one integral feasible flow is sufficient. Check every nonempty newly completed cluster using its actual counts and delete unused newly completed centers. Keep every inherited center in A, including centers with zero contribution in the current region. Then form the bucket vector of the inherited centers and keep one complete witness for each vector. Thus, compression changes which assignment is retained; it never changes the counts of a stored assignment. A center can be active for two reasons. It may have become active at the current separator, or it may already have been active from an earlier separator. Based on this, we make the following two observations.

▶ **Observation A.14.** *For every center $q \in Q$, after combining the inside, outside, and boundary assignments, the whole cluster of q is known. Therefore, its fairness can be checked, and q can be removed from the active set.*

▶ **Observation A.15.** *Let $a \in A$ be a center that became active at an earlier separator. At the current recursive step, we may see only part of the clients that will finally be assigned to a. Some other clients assigned to a may lie in another region created by that earlier separator. Therefore, the whole cluster of a is not known yet, so a must remain active.*

Algorithm 1 Group-Fair Euclidean k-Center in $\mathbb{R}^d$

If $P = \emptyset$, return the empty clustering. Set $k \leftarrow \min\{k, n\}$.
Construct the candidate-radius set $\mathcal{R}$ from all nonempty subsets of P of size at most $d+1$
for each $R \in \mathcal{R}$ in increasing order **do**
  if $R = 0$ **then**
    Test the zero-radius case described above; return if feasible, otherwise continue.
  end if
  Construct the graph G_R on P, where $pq \in E(G_R)$ if $\|p - q\|_2 \le 2R$.
  if G_R has more than k connected components **then**
    continue.
  end if
  if some connected component H satisfies $\mathrm{diam}(H) > (4k-2)R$ **then**
    continue.
  end if
  For every component, construct its enlarged bounding box.
  Set $\delta = \dfrac{\varepsilon R}{10\sqrt{d}}$ and use the common grid $\delta\mathbb{Z}^d$ in the padded enlarged boxes.
  Let F be the resulting set of candidate centers.
  Set $r = R + \dfrac{\delta\sqrt{d}}{2}$.
  for each $f \in F$ **do**
    Define the candidate ball $D_f = B(f, r)$.
  end for
  Compute the finite family of relevant separator-sphere types.
  $\mathcal{T} \leftarrow Sol(P, F, \emptyset, k)$.
  if $\mathcal{T}$ contains a feasible state with no active centers **then**
    Recover its integral clustering.
    return the clustering.
  end if
end for
return infeasible.

Algorithm 2 $Sol(P', F', A, \ell)$

if $\ell \leq 1$ **then**
  Execute the base case and **return** the resulting table.
end if
Initialize an empty table $\mathcal{T}$.
for each relevant separator sphere γ **do**
  Let $X_\gamma = \{f \in F' : D_f \text{ crosses } \gamma\}$.
  for each $Q \subseteq X_\gamma$ with $|Q| \leq \min\{\ell, w_d(\ell)\}$ **do**
    Set $A' \leftarrow A \cup Q$.
    Declare all centers in $X_\gamma \setminus Q$ unselected.
    Partition the clients into $P_{\text{in}}, P_{\text{out}}, P_\partial$.
    Let F_{in} and F_{out} be the noncrossing candidate centers lying completely inside and outside γ, respectively.
    for each pair $(\ell_{\text{in}}, \ell_{\text{out}})$ satisfying $\ell_{\text{in}}, \ell_{\text{out}} \leq \beta_d \ell$, and $|Q| + \ell_{\text{in}} + \ell_{\text{out}} \leq \ell$ **do**
      $\mathcal{T}_{\text{in}} \leftarrow \textsc{Solve}(P_{\text{in}}, F_{\text{in}}, A', \ell_{\text{in}})$.
      $\mathcal{T}_{\text{out}} \leftarrow \textsc{Solve}(P_{\text{out}}, F_{\text{out}}, A', \ell_{\text{out}})$.
      for each compatible pair of states $(\tau_{\text{in}}, \tau_{\text{out}})$ from $\mathcal{T}_{\text{in}}$ and $\mathcal{T}_{\text{out}}$ **do**
        Combine the contributions of the two states for all centers in A'.
        for each rounded bucket vector for the boundary contributions **do**
          Use the bucket-constrained flow to assign P_∂ to centers in A'.
          if the flow instance is infeasible **then**
            continue.
          end if
          Add the inside, outside, and boundary contributions to obtain the exact counts for every center in A'.
          Set valid $\leftarrow$ true.
          for each $q \in Q$ **do**
            Let X_q be its completed witness cluster; if $X_q = \emptyset$, omit q from this witness and continue.
            if for some $i \in [c]$, $\frac{|X_q \cap P_i|}{|X_q|} \notin \left[\frac{\alpha_i}{\lambda}, \min\{1, \lambda\beta_i\}\right]$ **then**
              valid $\leftarrow$ false.
            end if
          end for
          if valid = false **then**
            continue.
          end if
          Forget all newly completed centers in Q.
          Round the accumulated color counts of the inherited active centers in A.
          Insert the resulting state into $\mathcal{T}$, keeping one integral witness for each rounded state.
        end for
      end for
    end for
  end for
end for
return $\mathcal{T}$.

A.2 Correctness

After describing the algorithm, we now prove its correctness. We first handle the base case, where at most one new center can still be chosen, and then prove the recursive case for $\ell > 1$. The main idea is to show that if a correct fair solution exists, then at least one branch of the algorithm follows it and is never discarded.

Base Case: when $\ell \leq 1$. Consider a recursive call $Sol(P', F', A, \ell)$ with $\ell \leq 1$.

If $\ell = 0$, no new center can be selected. Hence, all clients in P' must be assigned to the already active centers in A.

If $\ell = 1$, the algorithm considers both possibilities: either no new center is selected, or exactly one center $f \in F'$ is selected. In the latter case, every possible choice of f is taken.

For each such choice, let A'' be the set of centers available for the assignment. Thus, $A'' = A$ if no new center is chosen, and $A'' = A \cup \{f\}$ if f is selected. The algorithm then enumerates the bucket values for the color counts contributed by the clients in P' and uses the flow construction to check whether these clients can be assigned to the centers in A''. For every feasible flow, test the actual completed cluster of the new center f, if it is nonempty, against the relaxed fairness bounds. Discard a witness failing this test, and delete f from this witness if it is unused. Inherited centers in A remain active. Insert each surviving witness under the bucket vector of its exact contributions to A, keeping one witness per vector. If both the client set and the available center set are empty, the empty assignment is feasible; if clients remain but no center is available, it is infeasible.

▶ **Lemma A.16.** *Suppose the current base-case call has a radius-r reference assignment using at most ℓ new centers, whose complete clusters at new centers are exactly (α, β)-fair. For inherited active centers, only their contributions from P' are prescribed. Then the algorithm finds a feasible state corresponding to this solution, up to the allowed rounding on the number of color counts.*

Proof. Choose the same zero-or-one new center as in the reference solution. For every center and color, place its exact count into the corresponding bucket. By Lemma A.13, the flow for these bucket choices is feasible. Any integral assignment returned for these same buckets has each color count within a factor θ of the corresponding reference count, while a zero count remains zero. By property 2, each color proportion therefore changes by at most a factor $\theta^2 \leq \lambda$. Hence every newly completed center satisfies the relaxed fairness condition. After table compression, the retained contribution to every inherited center differs from the reference contribution by at most θ^2: one factor for choosing a flow witness and one for keeping a representative with the same returned bucket vector. Zeros remain zero. No completed cluster is changed by this last operation; every retained witness has already passed its fairness test. ◀

Recursive Step: when $\ell > 1$

▶ **Lemma A.17.** *Suppose a call $Sol(P', F', A, \ell)$ has a radius-r reference assignment using at most $\ell > 1$ additional centers from F', with complete exactly fair clusters at these new centers. For inherited active centers, only the regional color contributions are prescribed. If this call has remaining recursion height h (a base case has height zero), its returned table contains an integral witness whose contribution to every inherited center-color pair is within a factor θ^{h+2} of the reference contribution, with zeros preserved. Every completed cluster of this witness is λ-relaxed fair.*

Proof. We prove the stated invariant by induction on the remaining recursion height. The base case follows from Lemma A.16 and its proof.

For $\ell > 1$, let $S \subseteq F'$ be the new centers used by the reference assignment, after deleting unused centers. If $S = \emptyset$, choose any representative sphere and take $Q = \emptyset$ and both child budgets zero. Otherwise, apply Theorem A.8 to the $|S| \leq \ell$ selected balls. Claim A.9 supplies an enumerated sphere with the same classifications. Choose Q to be precisely the crossing centers of S, and choose the child budgets to be the numbers of selected noncrossing centers

on the respective sides. These guesses satisfy $|Q| \le w_d(\ell)$, both child budgets are at most $\beta_d \ell$, and their sum with $|Q|$ is at most ℓ.

Restrict the reference assignment to the client sets defined by this representative sphere. Every noncrossing selected ball serves clients only on its own side. Thus, each child reference assignment completes exactly fair clusters at its own new centers, while centers in $A \cup Q$ remain inherited active centers. The induction hypothesis gives child witnesses whose active counts differ from these reference contributions by at most θ^{h+1}. Enumerate the buckets of the reference boundary contributions. Lemma A.13 supplies an integral boundary witness with distortion at most θ. By Observation A.11, adding the three contributions has distortion at most θ^{h+1}, with zeros preserved. For $q \in Q$, this sum is the complete cluster. The reference cluster is nonempty and exactly fair, so the witness cluster is also nonempty and passes the relaxed fairness test by Lemma A.12. All clusters completed within the children already passed this test and are kept unchanged. For inherited centers in A, keeping one witness with the same returned bucket vector introduces at most one further factor θ, by Observation A.10. This proves the bound θ^{h+2}. Even if many witnesses are inserted under this vector, the final witness and the reference-following candidate lie in the same bucket, so only one such factor is needed. Since $h \le D_d$, the total count distortion is at most θ^{D_d+2}, as used in the fairness lemma. ◀

Now comes our main theorem-

▶ **Theorem A.18.** *Fix $d \ge 2$, $c \ge 1$, $0 < \varepsilon \le 1$, and $\lambda > 1$. Let $P \subseteq \mathbb{R}^d$ be partitioned into c color classes with fairness bounds (α_i, β_i). If there exists an exactly (α, β)-fair clustering using at most k continuous centers, then the algorithm returns a clustering using at most k centers whose radius is at most $(1+\epsilon)R^\star$, where $R^\star$ is the optimum radius of an exactly fair solution. Moreover, every nonempty returned cluster X satisfies*

$$\frac{\alpha_i}{\lambda} \le \frac{|X \cap P_i|}{|X|} \le \min\{1, \lambda\beta_i\} \text{ for every } i \in [c].$$

Proof. The optimum radius $R^\star$ is among the radii considered by the algorithm. If $R^\star = 0$, the separate zero-radius test returns an exactly fair solution. Hence assume $R^\star > 0$. For this radius, Lemma A.5 gives an exactly fair solution using candidate centers from F, with radius $r < (1+\epsilon)R^\star$. Now consider the root call $Sol(P, F, \emptyset, k)$. Lemma A.16 shows that a correct branch is preserved in every base case. For a recursive call with $\ell > 1$, Lemma A.17 shows that the algorithm considers a separator, a set of selected crossing centers, and inside and outside budgets that agree with the exact fair solution. Therefore, by induction over the recursion, at least one feasible branch survives up to the root. Every client in this branch is assigned exactly once, either in one of the recursive subproblems or through the boundary flow. An assignment is allowed only if the client is within distance r of its assigned center. The center-budget conditions also ensure that at most k centers are selected.

Whenever the complete cluster of a center becomes known, its fairness is checked. These tests use the actual integral witness counts. Later compression only chooses between already valid witnesses and never modifies a completed cluster, so every returned witness is sound. For the witness supplied by the completeness proof, Lemma A.12 ensures that these tests succeed. Hence every nonempty completed cluster satisfies

$$\frac{\alpha_i}{\lambda} \le \frac{|X \cap P_i|}{|X|} \le \min\{1, \lambda\beta_i\} \text{ for every } i \in [c].$$

At the root, there are no remaining active centers, so the surviving branch gives a complete clustering. It uses at most k centers, has radius at most $r < (1+\varepsilon)R^\star$, and satisfies the

required relaxed fairness condition for every cluster. If the algorithm returns earlier at a candidate radius $R_0 < R^\star$, the same soundness argument applies and its radius is at most $(1+\varepsilon)R_0 \le (1+\varepsilon)R^\star$. ◀

A.3 Runtime Analysis

We first recall the two bounds needed for the running-time analysis. Since each recursive child has center budget at most $\beta_d \ell$, where $\beta_d = \frac{d+1}{d+2} < 1$, the recursion depth is $D_d = \left\lceil \log_{1/\beta_d} k \right\rceil + 1 = \mathcal{O}_d(\log(k+1))$. Moreover, at a level with budget ℓ, at most $\mathcal{O}_d\left(\frac{\ell^{1-1/d}}{\varepsilon}\right)$ new crossing centers become active. Summing this along one root-to-leaf path gives $H_d = \mathcal{O}_d\left(\frac{k^{1-1/d}}{\varepsilon}\right)$. Thus, the recursion has depth $\mathcal{O}_d(\log k)$, and at any time at most H_d centers need to be kept active. We now count the different contributions to the running time.

Radius Enumeration. By the radius-enumeration step in Section 4, the algorithm considers $\mathcal{O}(n^{d+1})$ candidate radii. For fixed d, this is $n^{\mathcal{O}(d)}$.

Candidate Centers. By Lemma A.4, the number of grid candidate centers is

$$M = \mathcal{O}\left(\frac{k^{d+1} d^{d/2}}{\varepsilon^d}\right) = \left(\frac{k}{\varepsilon}\right)^{\mathcal{O}(d)} \quad \text{for fixed } d.$$

Separator Types. Claim A.9 gives $M^{\mathcal{O}(d)}$ representative separator types at one recursive node. Since the recursion depth is at most D_d, their contribution along a recursive path is $M^{\mathcal{O}_d(D_d)}$.

Crossing-Center Guesses. At a node with budget ℓ, at most $w_d(\ell) = \mathcal{O}_d\left(\frac{\ell^{1-1/d}}{\varepsilon}\right)$ selected balls cross the separator. Hence the number of possible choices for the selected crossing set Q is at most $\sum_{j=0}^{w_d(\ell)} \binom{M}{j} \le M^{\mathcal{O}_d(\ell^{1-1/d}/\varepsilon)}$. Along one root-to-leaf path, these exponents sum to $\mathcal{O}_d(H_d)$. Therefore, the total contribution of the crossing-set guesses is $M^{\mathcal{O}_d(H_d)}$. The two recursive children are solved separately, so their running times are added. The cost of combining their DP tables is included in the state-table bound below.

For clarity, let $B(\ell)$ be the number of separator, crossing-set, and budget guesses at a node. Then $B(\ell) \le M^{\mathcal{O}(d)}(k+1)^2 M^{\mathcal{O}_d(\ell^{1-1/d}/\varepsilon)}$. If W is a uniform bound on local table processing, the recursive cost satisfies $T(\ell) \le B(\ell)(2T(\lfloor \beta_d \ell \rfloor) + W)$ for $\ell > 1$, with the base-case cost included in W. Unrolling this bound counts all calls by the product of these branching factors along a path, with an additional $2^{\mathcal{O}(D_d)}$ factor. Local table processing is performed once per call; its state factor is not multiplied once per recursion level.

Budget guesses. At each recursive level, there are at most $\mathcal{O}(k^2)$ choices for $(\ell_{\text{in}}, \ell_{\text{out}})$. Over depth D_d, this contributes $k^{\mathcal{O}(D_d)}$.

Rounded DP states. Each active center has c color-count coordinates, and each coordinate has at most L possible buckets. Since at most $H_d + 1$ centers are active, the number of rounded count states is at most $L^{c(H_d+1)}$. Combining child tables and enumerating the bucket vectors used by the flow step changes only the constant in the exponent. Hence the table-processing cost is $L^{\mathcal{O}(cH_d)} k^{\mathcal{O}(1)}$. Each individual flow computation takes polynomial time. The base case considers at most $M+1$ choices for its zero-or-one new center; this additional factor is absorbed into the power of M below.

Combining all these factors gives $T_{c,d} = n^{\mathcal{O}(d)} M^{\mathcal{O}_d(H_d+D_d)} L^{\mathcal{O}(cH_d)} k^{\mathcal{O}(D_d)}$, where

$$H_d = \mathcal{O}_d\left(\frac{k^{1-1/d}}{\epsilon}\right),\ D_d = \mathcal{O}_d(\log(k+1)),\ \text{and}\ L = \mathcal{O}\left(1 + \frac{D_d \log n}{\log \lambda}\right).$$

The factor involving $\log n$ can be absorbed into a polynomial factor in n together with a parameter-dependent factor.

Indeed, for $a \geq 1$ and $n \geq 1$, $(1+\log n)^a \leq n(a+1)^a$. Taking $a = \mathcal{O}(c(H_d+1))$ and using $L = \mathcal{O}((1 + D_d/\log\lambda)(1+\log n))$ gives $L^a \leq n[\mathcal{O}((1 + D_d/\log\lambda)(a+1))]^a$. The exponent of n remains independent of k. Therefore, for fixed d,

$$T_{c,d} = n^{\mathcal{O}(d)} \left(\frac{k}{\varepsilon}\right)^{\mathcal{O}_d(k^{1-1/d}/\varepsilon)} \left(\frac{c\,k^{1-1/d}}{\varepsilon}\left(1 + \frac{\log(k+1)}{\log\lambda}\right)\right)^{\mathcal{O}_d(c\,k^{1-1/d}/\varepsilon)}.$$

In particular, $T_{c,d} = n^{\mathcal{O}(d)} \left(\frac{k}{\varepsilon}\right)^{\mathcal{O}_{c,d,\lambda}(k^{1-1/d}/\varepsilon)}$.